

Kiểm tra 2

1.

1.1. Lịch sử phát triển

* CNN

+ Lịch sử phát triển

CNN ra đời từ thập niên 1980 và trở nên nổi bật nhờ công trình của Yann LeCun vào đầu thập niên 1990. Trong nghiên cứu nhận dạng chữ viết tay (đặc biệt là mã bưu điện ZIP Code), ông cùng cộng sự đã phát triển LeNet-5 – một trong những kiến trúc CNN đầu tiên, đặt nền móng cho lĩnh vực thị giác máy tính.

Đến năm 2012, sự ra đời của AlexNet đánh dấu bước ngoặt lớn khi giành chiến thắng trong cuộc thi ImageNet, giúp CNN trở thành nền tảng cho các ứng dụng xử lý hình ảnh hiện đại. Sau đó, nhiều mô hình cải tiến ra đời như VGG, Inception và ResNet, giúp nâng cao độ chính xác và khả năng học sâu của mạng.

+ Kiến trúc CNN

CNN được thiết kế để tự động học và trích xuất đặc trưng không gian trong ảnh thông qua ba lớp chính:

- Lớp Tích chập (Convolutional Layer): Dùng các bộ lọc (filter) quét qua ảnh để trích xuất đặc trưng như cạnh, góc, hay hình dạng phức tạp.
- Lớp Gộp (Pooling Layer): Giảm kích thước dữ liệu và loại bỏ nhiễu thông qua các phương pháp như *max pooling* hoặc *average pooling*.
- Lớp Kết nối Đầy đủ (Fully Connected Layer): Kết hợp các đặc trưng đã học để thực hiện các tác vụ như phân loại hoặc hồi quy.

Nhờ cấu trúc này, CNN trở thành trụ cột của các ứng dụng như nhận diện khuôn mặt, xe tự lái, và chẩn đoán hình ảnh y học.

* RNN

+ Lịch sử phát triển

RNN xuất hiện từ những năm 1980 để xử lý dữ liệu tuần tự – nơi thông tin ở thời điểm hiện tại phụ thuộc vào các bước trước đó (ví dụ: văn bản, chuỗi thời gian). Tuy nhiên, RNN sớm gặp vấn

đề vanishing gradient và exploding gradient, khiến việc học với chuỗi dài trở nên khó khăn. Sau này, kỹ thuật Backpropagation Through Time (BPTT) ra đời giúp cải thiện quá trình huấn luyện, mở đường cho các biến thể mạnh mẽ hơn như LSTM và GRU.

+ Đặc trưng và Kiến trúc

RNN có khả năng “ghi nhớ” trạng thái ẩn qua từng bước thời gian:

- Lớp Ẩn (Hidden Layer): Ở mỗi bước, đầu ra phụ thuộc vào đầu vào hiện tại và trạng thái ẩn trước đó.
- Chuỗi Thời gian: Trạng thái ẩn được duy trì và cập nhật, giúp RNN hiểu được ngữ cảnh và quan hệ giữa các phần tử trong chuỗi.

Nhờ đặc tính này, RNN được ứng dụng trong xử lý ngôn ngữ tự nhiên (dịch máy, tóm tắt văn bản, nhận dạng giọng nói) và dự báo chuỗi thời gian.

* LSTM

+ Lịch sử phát triển

LSTM được giới thiệu năm 1997 bởi Sepp Hochreiter và Jürgen Schmidhuber nhằm khắc phục hạn chế lớn của RNN – hiện tượng mất hoặc nổ gradient. Với cơ chế cổng kiểm soát thông tin, LSTM có khả năng lưu trữ và quên thông tin linh hoạt, giúp học được các quan hệ dài hạn trong dữ liệu chuỗi.

Ngày nay, LSTM là thành phần cốt lõi trong nhiều ứng dụng hiện đại như dịch máy, phân tích cảm xúc, nhận dạng giọng nói và dự báo dữ liệu thời gian dài.

Kiến trúc và Cơ chế hoạt động

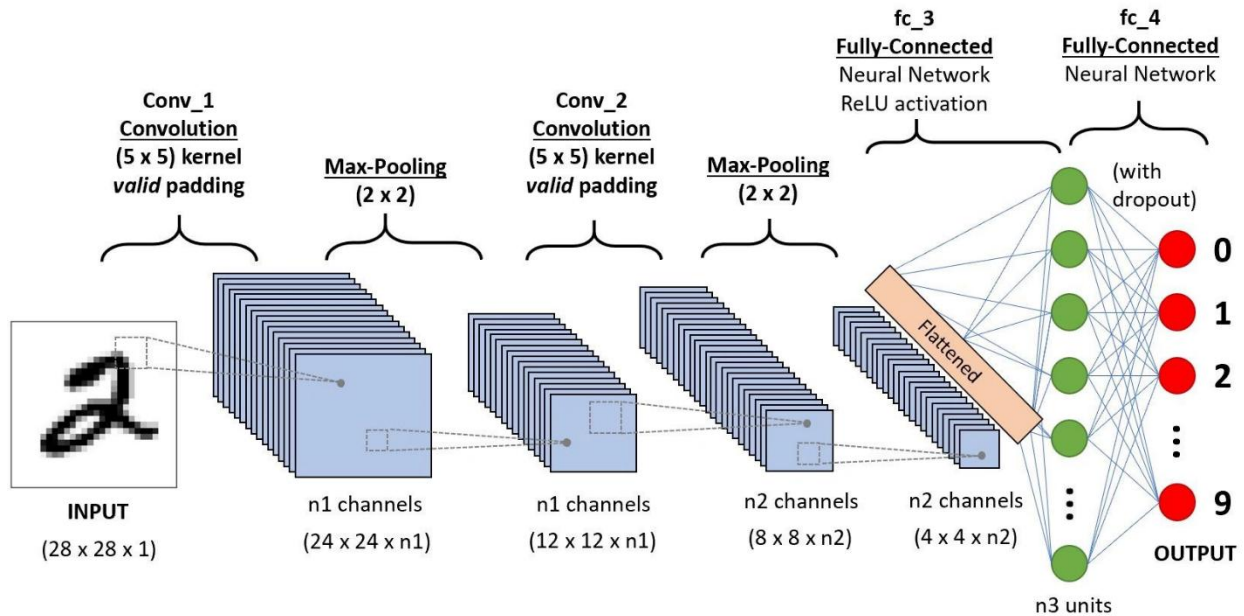
LSTM mở rộng RNN bằng ba cổng chính:

- Forget Gate (Cổng Quên): Xác định thông tin nào trong trạng thái cũ cần loại bỏ.
- Input Gate (Cổng Nhập): Kiểm soát lượng thông tin mới được lưu vào bộ nhớ.
- Output Gate (Cổng Xuất): Quyết định phần thông tin nào sẽ được xuất ra ngoài.

Nhờ các cổng này, LSTM có thể ghi nhớ thông tin trong thời gian dài, duy trì sự ổn định trong quá trình học, và hoạt động hiệu quả hơn trên các chuỗi dữ liệu phức tạp.

1.2. Kiến trúc

* CNN



- **Lớp Tích chập (Convolutional Layer)**

- Đây là khối xây dựng chủ đạo của CNN. Mỗi lớp tích chập sử dụng một tập các bộ lọc (filters/kernel) có thể học được, quét dọc theo chiều rộng và chiều cao của ảnh đầu vào (và xuyên chiều sâu nếu có).
- Mỗi bộ lọc có vùng tiếp nhận (receptive field) nhỏ nhưng “vươn” xuyên toàn bộ chiều sâu của đầu vào. Khi quét, nó tính tích vô hướng (dot product) giữa bộ lọc và vùng tương ứng của đầu vào, tạo ra một “bản đồ kích hoạt” (activation map) hoặc feature map.
- Kết quả là nhiều bản đồ kích hoạt (một cho mỗi bộ lọc) được chồng lên nhau theo chiều sâu, hình thành đầu ra của lớp tích chập.

- **Lớp Kích hoạt (Activation Layer – thường dùng ReLU)**

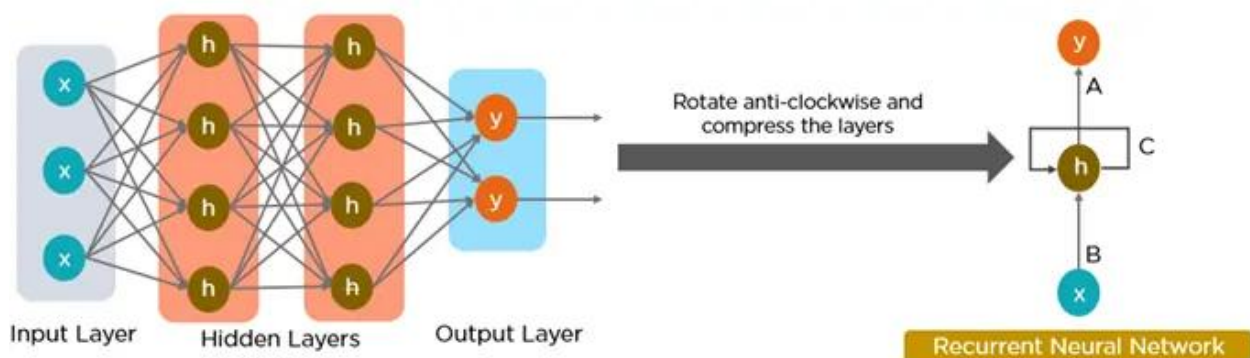
- Sau mỗi lớp tích chập thường được thêm một lớp kích hoạt phi tuyến (non-linearity). Ví dụ: hàm ReLU (Rectified Linear Unit) là phổ biến vì giúp huấn luyện mạng sâu tốt hơn.

- **Lớp Gộp (Pooling Layer)**

- Được sử dụng để giảm kích thước không gian (chiều rộng, chiều cao) của các bản đồ đặc trưng, giúp giảm số lượng tham số và độ phức tạp tính toán.

- Thường dùng *max-pooling* hoặc *average-pooling*. Ví dụ: chia bản đồ đặc trưng thành các vùng nhỏ (ví dụ 2×2) rồi lấy giá trị lớn nhất hoặc trung bình mỗi vùng.
 - Việc gộp cũng giúp mạng có khả năng bất biến dịch chuyển cục bộ (local translation invariance) — tức là nếu vật thể hơi dịch chuyển trong ảnh, mạng vẫn có thể nhận diện được.
- **Chuỗi lặp lại các khối trên + có thể thêm lớp chuẩn hóa (normalization) hoặc dropout**
- Trong thực tế, một CNN sẽ lồng nhiều lần khối (Convolution → Activation → Pooling) để tự học từ đặc trưng đơn giản tới phức tạp hơn.
 - Có thể thêm các lớp như Batch Normalization, Dropout để tăng hiệu quả học và giảm over-fitting (mặc dù bài viết không đi sâu vào tất cả các biến thể).
- **Lớp kết nối đầy đủ (Fully Connected Layer)**
- Sau khi các đặc trưng không gian đã được trích xuất và thu nhỏ bởi các lớp tích chập và pooling, kết quả thường được *flatten* (dàn mỏng) và nối với các lớp kết nối đầy đủ (các neuron nối đầy đủ với nhau) để thực hiện tác vụ cuối cùng như phân loại (classification) hoặc hồi quy (regression).
 - Cuối cùng có thể dùng hàm softmax (hoặc sigmoid, tùy bài toán) để đưa ra xác suất cho các lớp.

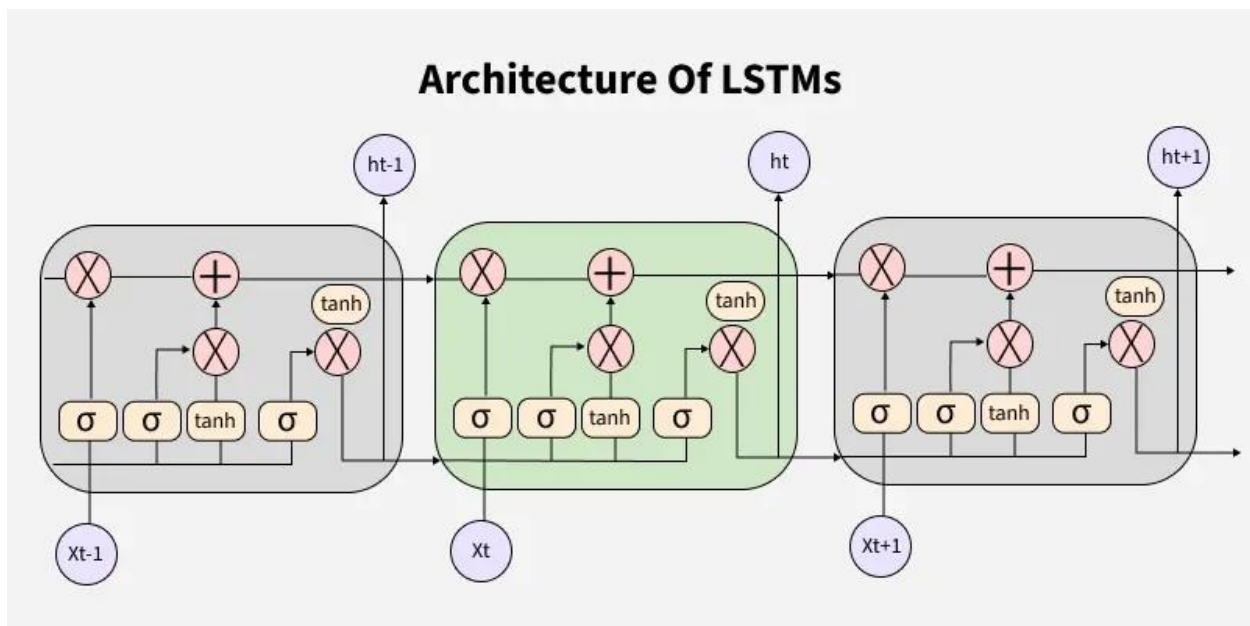
* RNN



- RNN được thiết kế để xử lý dữ liệu theo chuỗi, nghĩa là thứ tự của các phần tử (như từ trong câu, bước thời gian trong chuỗi) quan trọng.
- Khác với mạng truyền thẳng thông thường (Feed-forward), RNN có các trạng thái ẩn (hidden state) lưu giữ thông tin từ các bước trước để sử dụng ở bước hiện tại.

- Cấu trúc cơ bản: tại mỗi bước thời gian t , input x_t và trạng thái ẩn trước đó h_{t-1} được dùng để tính trạng thái ẩn mới h_t . Sau đó h_t có thể dùng để tính đầu ra y_t .
- RNN có thể được triển khai với nhiều kiểu đầu vào/đầu ra:
 - one-to-one (giống mạng thông thường)
 - one-to-many (một input, nhiều output)
 - many-to-one (nhiều input, một output)
 - many-to-many (chuỗi input $\rightarrow$ chuỗi output)

* LSTM



- LSTM là một biến thể của mạng hồi tiếp (RNN) được thiết kế để khắc phục vấn đề gradient biến mất (vanishing gradient) và gradient bùng nổ (exploding gradient) trong mạng RNN truyền thống.
- Mỗi cell LSTM gồm một “bộ nhớ” gọi là cell state và các cổng (gates) để điều khiển luồng thông tin: nhập vào, quên đi, và xuất ra.
- Các cổng chính gồm:
 - Forget gate (Cổng quên): Quyết định phần thông tin nào trong trạng thái cũ cần bị loại bỏ.
 - Input gate (Cổng nhập): Kiểm soát thông tin mới từ input và trạng thái trước được đưa vào cell state.
 - Output gate (Cổng xuất): Quyết định phần thông tin từ cell state nào sẽ được đưa tới trạng thái ẩn (hidden state) và đầu ra hiện tại.

1.3. Code

* CNN

```
[*]: # Nguyễn Hoàng Tùng

# =====
# CNN cơ bản để phân loại ảnh MNIST (TensorFlow)
# =====

import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt

# 1 Tải và tiền xử lý dữ liệu
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

# Chuyển thành ảnh 28x28x1 (1 kênh màu xám)
x_train = x_train.reshape((x_train.shape[0], 28, 28, 1)).astype('float32') / 255
x_test = x_test.reshape((x_test.shape[0], 28, 28, 1)).astype('float32') / 255
```

```
# 2 Xây dựng kiến trúc CNN
model = models.Sequential([
    # Lớp tích chập 1: 32 bộ lọc 3x3
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),

    # Lớp tích chập 2: 64 bộ lọc 3x3
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Lớp tích chập 3: 64 bộ lọc 3x3
    layers.Conv2D(64, (3, 3), activation='relu'),

    # Flatten để đưa sang fully connected
    layers.Flatten(),

    # Fully connected 64 neuron
    layers.Dense(64, activation='relu'),

    # Lớp output 10 lớp tương ứng 10 chữ số
    layers.Dense(10, activation='softmax')
])

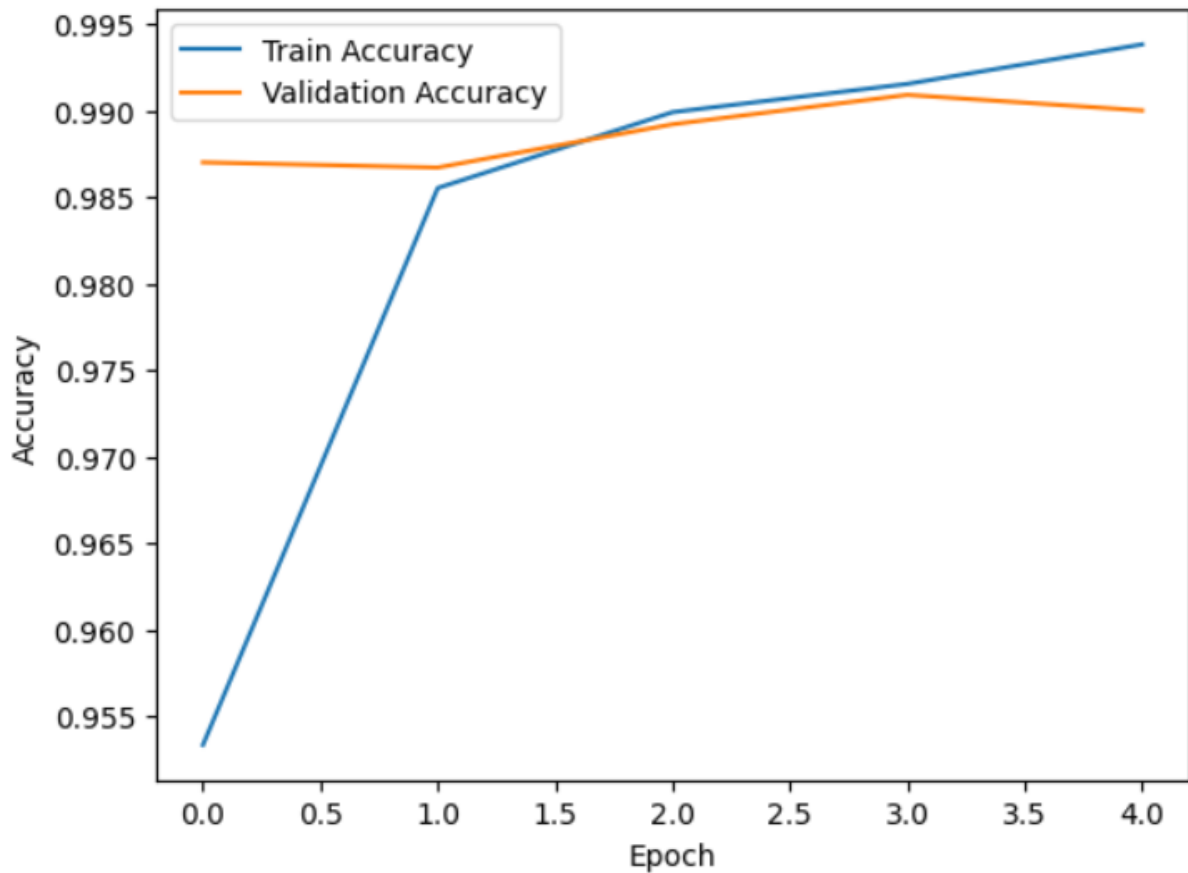
# 3 Biên dịch mô hình
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# 4 Huấn luyện mô hình
history = model.fit(x_train, y_train, epochs=5,
                   validation_data=(x_test, y_test))
```

```
# 5 Đánh giá mô hình
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"\n✅ Độ chính xác trên tập kiểm tra: {test_acc*100:.2f}%")

# 6 (Tùy chọn) Vẽ biểu đồ huấn luyện
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

✓ Độ chính xác trên tập kiểm tra: 99.00%



* RNN

```
[*]: # Nguyễn Hoàng Tùng

# =====
# RNN cơ bản (SimpleRNN) phân loại chữ số MNIST
# =====

import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt

# 1 Tải dữ liệu MNIST
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

# Chuẩn hóa dữ liệu về [0,1]
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# 2 Xây dựng mô hình RNN
model = models.Sequential([
    # Mỗi ảnh 28x28 được xem như chuỗi 28 bước, mỗi bước 28 giá trị
    layers.SimpleRNN(128, activation='tanh', input_shape=(28, 28)),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

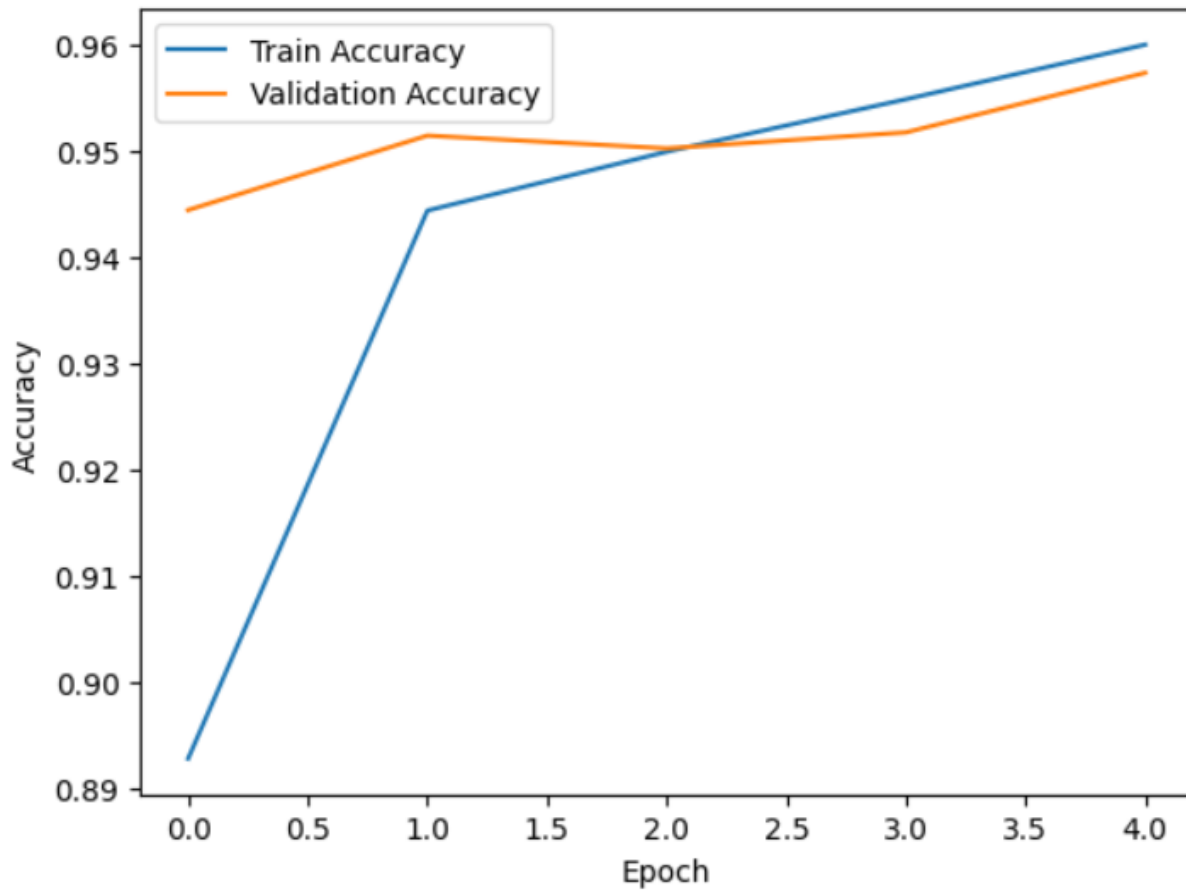
# 3 Biên dịch mô hình
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```
# 4 Huấn Luyện mô hình
history = model.fit(x_train, y_train, epochs=5,
                    validation_data=(x_test, y_test))

# 5 Đánh giá mô hình
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"\n✅ Độ chính xác trên tập kiểm tra: {test_acc*100:.2f}%")

# 6 (Tuỳ chọn) Vẽ biểu đồ độ chính xác
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

✅ Độ chính xác trên tập kiểm tra: 95.73%

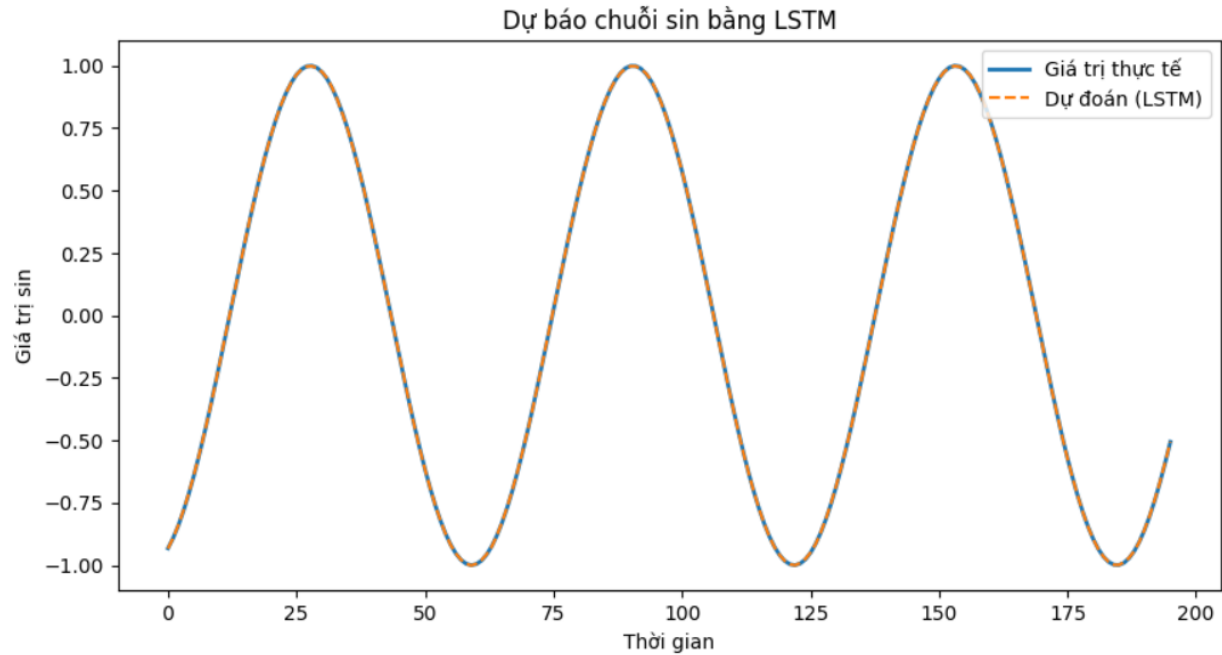


* LSTM

[3]: # Nguyễn Hoàng Tùng

```
# =====  
# LSTM cơ bản - Dự đoán giá trị tiếp theo trong chuỗi sin  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import LSTM, Dense  
  
# 1 Tạo dữ liệu chuỗi sin  
x = np.linspace(0, 100, 1000)  
y = np.sin(x)  
  
# Chuẩn bị dữ liệu huấn luyện: chia chuỗi thành các đoạn nhỏ (window)  
time_steps = 20  
  
def create_dataset(series, time_steps=1):  
    X, Y = [], []  
    for i in range(len(series) - time_steps):  
        X.append(series[i:(i + time_steps)])  
        Y.append(series[i + time_steps])  
    return np.array(X), np.array(Y)  
  
X, Y = create_dataset(y, time_steps)  
  
# Thêm 1 chiều cho LSTM: (samples, time_steps, features)  
X = X.reshape((X.shape[0], X.shape[1], 1))
```

```
# 2 Chia tập huấn luyện & kiểm thử  
split = int(0.8 * len(X))  
X_train, X_test = X[:split], X[split:]  
Y_train, Y_test = Y[:split], Y[split:]  
  
# 3 Xây dựng mô hình LSTM  
model = Sequential([  
    LSTM(64, activation='tanh', input_shape=(time_steps, 1)),  
    Dense(32, activation='relu'),  
    Dense(1)  
])  
  
# 4 Biên dịch và huấn luyện  
model.compile(optimizer='adam', loss='mse')  
history = model.fit(X_train, Y_train, epochs=20, batch_size=16,  
                    validation_data=(X_test, Y_test), verbose=1)  
  
# 5 Dự đoán  
Y_pred = model.predict(X_test)  
  
# 6 Vẽ kết quả  
plt.figure(figsize=(10,5))  
plt.plot(Y_test, label='Giá trị thực tế', linewidth=2)  
plt.plot(Y_pred, label='Dự đoán (LSTM)', linestyle='--')  
plt.legend()  
plt.title("Dự báo chuỗi sin bằng LSTM")  
plt.xlabel("Thời gian")  
plt.ylabel("Giá trị sin")  
plt.show()  
|
```



1.4. Sự khác biệt giữa 3 mô hình

Mô hình	Đặc điểm chính	Ứng dụng tiêu biểu
CNN	Trích xuất đặc trưng không gian, xử lý ảnh	Nhận diện hình ảnh, thị giác máy tính
RNN	Xử lý dữ liệu tuần tự, có khả năng ghi nhớ ngắn hạn	Ngôn ngữ tự nhiên, dự báo chuỗi thời gian
LSTM	Ghi nhớ dài hạn, khắc phục hạn chế của RNN	Dịch máy, phân tích cảm xúc, nhận dạng giọng nói

2.

2.1.

```
[4]: # Nguyễn Hoàng Tùng

import pandas as pd

# Đường dẫn tới thư mục chứa các file dataset
path = r'D:\PTIT\20251\IntSys'

# Đọc 3 file dataset
df1 = pd.read_csv(f'{path}\\df_weather.csv')
df2 = pd.read_csv(f'{path}\\precipitation_mean_max_min_temp_1901_2020_en.csv')
df3 = pd.read_csv(f'{path}\\weather.csv')

# Hiển thị thông tin cơ bản về từng dataset
print("=== Dataset 1: df_weather.csv ===")
display(df1.head())
print(df1.info(), "\n")

print("=== Dataset 2: precipitation_mean_max_min_temp_1901_2020_en.csv ===")
display(df2.head())
print(df2.info(), "\n")

print("=== Dataset 3: weather.csv ===")
display(df3.head())
print(df3.info())
```

2.2. Ba pha phát triển mô hình học sâu

Pha 1. Xây dựng mô hình có khả năng khái quát và có thể overfit

- Mục tiêu: tạo ra một mô hình có thể học được dữ liệu (fit) và thể hiện dấu hiệu overfitting (quá khớp).
- Các bước:
 - o Điều chỉnh learning rate và batch size.
 - o Tăng model capacity (số lớp, số neuron).
 - o Chọn kiến trúc phù hợp hơn (CNN, RNN, Transformer,...).
 - o Huấn luyện mô hình lâu hơn.
- Kết quả mong đợi: mô hình có thể học tốt trên tập huấn luyện và có khả năng tổng quát ban đầu.

Pha 2. Cải thiện khả năng tổng quát (generalization)

- Khi mô hình bắt đầu overfit (hiệu suất huấn luyện cao nhưng kiểm thử kém), cần tập trung giảm overfitting bằng:
 - o Giảm độ phức tạp mô hình (số lớp, số neuron).
 - o Thêm regularization: Dropout, L2/L1 weight regularization.
 - o Early stopping: dừng huấn luyện khi loss validation tăng.
 - o Tăng dữ liệu huấn luyện hoặc cải thiện chất lượng dữ liệu.
- Kết quả mong đợi: mô hình tổng quát tốt hơn trên dữ liệu chưa thấy.

Pha 3. Đánh giá và triển khai (model evaluation & deployment)

- Đánh giá mô hình bằng các tập:
 - o Training set để huấn luyện.

- Validation set để điều chỉnh hyperparameter.
- Test set để kiểm tra tổng quát hóa cuối cùng.
- Nếu đạt kết quả tốt, mô hình có thể được triển khai, giám sát và bảo trì.
- Kết quả mong đợi: mô hình đáng tin cậy, sẵn sàng áp dụng trong thực tế.

Áp dụng 3 pha này vào bài toán dự báo thời tiết Việt Nam

Pha	Mô tả trong bối cảnh dự báo thời tiết	Ví dụ cụ thể
1. Xây dựng mô hình cơ bản và kiểm tra khả năng học	Thu thập và chuẩn hóa dữ liệu thời tiết (nhiệt độ, độ ẩm, lượng mưa, gió,...). Xây dựng mô hình thử nghiệm (LSTM, GRU hoặc CNN-1D).	Huấn luyện mô hình LSTM trên dữ liệu 10 năm để dự báo nhiệt độ ngày hôm sau.
2. Cải thiện tổng quát hóa	Khi mô hình dự báo tốt trên train nhưng kém trên validation, áp dụng kỹ thuật regularization, dropout, hoặc tăng dữ liệu bằng cách trượt cửa sổ thời gian (sliding window).	Thêm Dropout=0.3 trong LSTM, mở rộng dữ liệu bằng trung bình 3 ngày gần nhất, hoặc thêm dữ liệu khí tượng khu vực lân cận.
3. Đánh giá và triển khai	Đánh giá mô hình bằng tập test độc lập (ví dụ: dữ liệu của năm 2024). Sau đó triển khai vào hệ thống cảnh báo thời tiết.	Mô hình dự báo được triển khai trên API Flask, nhận dữ liệu khí tượng thực tế và xuất dự báo tự động.

2.3.

[7]: # Nguyễn Hoàng Tùng

```
# Notebook: RNN & LSTM time-series forecasting for Hanoi, Danang, HCM
# Run in Jupyter. Make sure tensorflow is installed: pip install tensorflow

import os
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# ----- CONFIG -----
# Thay đổi đường dẫn nếu cần (nếu chạy Local Windows, bạn có thể set DATA_DIR = r"D:\PTIT\20251\IntSys")
DATA_DIRS = [Path("/mnt/data"), Path(r"D:\PTIT\20251\IntSys")]
# Tên file (nếu khác, sửa ở đây)
FILES = ["df_weather.csv", "precipitation_mean_max_min_temp_1901_2020_en.csv", "weather.csv"]

TARGET_CITIES = ["Hà Nội", "Đà Nẵng", "Hồ Chí Minh"] # tên hiển thị
LOOKBACK = 14 # dùng 14 ngày trước để dự đoán 1 ngày tới
BATCH_SIZE = 32
MAX_EPOCHS = 30

# ----- try import tensorflow -----
try:
    import tensorflow as tf
    from tensorflow import keras
    from tensorflow.keras import layers
except Exception as e:
    raise RuntimeError("TensorFlow không được tìm thấy. Cài bằng `pip install tensorflow` rồi chạy lại.") from e
```

```
# ----- helper: find and load CSVs -----
def find_files():
    found = {}
    for d in DATA_DIRS:
        if not d.exists():
            continue
        for fname in FILES:
            p = d / fname
            if p.exists():
                found[fname] = p
    return found

found = find_files()
if not found:
    print("Không tìm thấy file trong các đường dẫn mặc định. Kiểm tra DATA_DIRS / FILES.")
else:
    print("Tìm thấy:", found)

# Load available files into dataframes
dfs = {}
for fname, path in found.items():
    try:
        df = pd.read_csv(path)
    except Exception:
        df = pd.read_csv(path, encoding="latin1")
    dfs[fname] = df
    print(f"\nLoaded {fname} shape={df.shape}")
    print(df.columns.tolist()[:20])
```

```
# ----- heuristic: extract per-city temperature series -----
# We'll attempt to locate columns: date, city, temperature-like.
def find_temp_col(cols):
    cols_low = [c.lower() for c in cols]
    # prefer column containing 'temp' but not 'min'/'max' (for avg)
    for i, c in enumerate(cols_low):
        if "temp" in c and "max" not in c and "min" not in c:
            return cols[i]
    # fallback common names
    for cand in ("temperature", "tavg", "t_mean", "t"):
        for i, c in enumerate(cols_low):
            if cand in c:
                return cols[i]
    return None

city_series = {} # map display_name -> pd.Series(index=date, values=temp)
```

```

# Try each loaded df to find city/time/temp
for name, df in dfs.items():
    cols = df.columns.tolist()
    date_col = None
    city_col = None
    # find date column
    for c in cols:
        if "date" in c.lower() or "time" in c.lower() or "day" in c.lower():
            date_col = c
            break
    # find city column
    for c in cols:
        if any(x in c.lower() for x in ("city", "station", "location", "name")):
            city_col = c
            break
    temp_col = find_temp_col(cols)
    # If file is pivoted (column per city), detect and use directly
    if not city_col and any(any(tc.lower() in c.lower() for tc in ["hanoi", "ha noi", "hà nội", "ha noi", "da nang", "đà nẵng", "ho chi minh", "hcm", "hochiminh"])
        # find columns that match cities and create series (assume index is date or range)
        if date_col:
            df[date_col] = pd.to_datetime(df[date_col], errors="coerce")
            for city in TARGET_CITIES:
                # try match by substring
                for c in cols:
                    if city.split()[0].lower()[0:4] in c.lower():
                        ser = df.set_index(date_col)[c].dropna().sort_index()
                        city_series[city] = ser.astype(float)
                else:
                    # no date; skip this approach
                    pass
            elif date_col and city_col and temp_col:
                df2 = df[[date_col, city_col, temp_col]].copy()
                df2[date_col] = pd.to_datetime(df2[date_col], errors="coerce")
                df2 = df2.dropna(subset=[date_col, city_col, temp_col])
                for city in TARGET_CITIES:
                    mask = df2[city_col].astype(str).str.lower().str.contains(city.split()[0].lower()[0:4])
                    if mask.any():
                        ser = df2.loc[mask].set_index(date_col)[temp_col].sort_index()
                        city_series[city] = ser.astype(float)

```

```

# If still missing, try scanning columns names in all dfs for city substrings
if not city_series:
    for name, df in dfs.items():
        for c in df.columns:
            for city in TARGET_CITIES:
                if city.split()[0].lower()[0:4] in str(c).lower():
                    # try to use column as series; need index
                    if 'date' in (col.lower() for col in df.columns):
                        datecol = [col for col in df.columns if 'date' in col.lower()][0]
                        s = pd.Series(df[c].values, index=pd.to_datetime(df[datecol], errors="coerce"))
                        city_series[city] = s.dropna().astype(float)

if not city_series:
    raise RuntimeError("Không tự động trích xuất được chuỗi nhiệt độ cho các thành phố. Bạn hãy chỉ rõ cột date/city/temp hoặc cung cấp file per-city.")

print("\nCities found for modeling:", list(city_series.keys()))

# ----- prepare supervised dataset from series -----
def prepare_series(series, lookback=LOOKBACK):
    s = series.dropna().sort_index()
    # Nếu có duplicate index (nhiều giá trị cùng ngày), lấy trung bình theo ngày
    if isinstance(s.index, pd.DatetimeIndex):
        if not s.index.is_unique:
            s = s.groupby(s.index).mean()
            # reindex về tần suất ngày, điền giá trị còn thiếu
            s = s.asfreq('D')
            s = s.interpolate(limit_direction='both')
    arr = s.values.astype(np.float32)
    X, y = [], []
    for i in range(lookback, len(arr)):
        X.append(arr[i - lookback:i])
        y.append(arr[i]) # dự đoán ngày tiếp theo
    X = np.array(X)[1:, :, None] # shape (samples, lookback, 1)
    y = np.array(y)
    return X, y, s

def split_dataset(X, y, train_pct=0.7, val_pct=0.15):
    n = len(X)
    n_train = int(n * train_pct)
    n_val = int(n * (train_pct + val_pct))
    return (X[:n_train], y[:n_train]), (X[n_train:n_val], y[n_train:n_val]), (X[n_val:], y[n_val:])

```

```
# ----- model builders -----
def build_simple_rnn(input_shape):
    inp = keras.Input(shape=input_shape)
    x = layers.SimpleRNN(32)(inp)
    x = layers.Dense(8, activation='relu')(x)
    out = layers.Dense(1)(x)
    model = keras.Model(inp, out)
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])
    return model

def build_lstm(input_shape):
    inp = keras.Input(shape=input_shape)
    x = layers.LSTM(32)(inp)
    x = layers.Dense(8, activation='relu')(x)
    out = layers.Dense(1)(x)
    model = keras.Model(inp, out)
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])
    return model
```

```
# ----- training loop per city -----
results = {}
models_dir = Path.cwd() / "weather_models"
models_dir.mkdir(exist_ok=True)

for city, series in city_series.items():
    print(f"\n--- Processing city: {city} ---")
    X, y, full_series = prepare_series(series, LOOKBACK)
    if len(X) < 80:
        print(f"    (Dữ liệu ít: only {len(X)} samples) - recommend collect more history. Skipping training for {city}.")
        continue
    (X_train, y_train), (X_val, y_val), (X_test, y_test) = split_dataset(X, y)
    # normalize based on train
    mean = X_train.mean()
    std = X_train.std()
    X_train_n = (X_train - mean) / (std + 1e-6)
    X_val_n = (X_val - mean) / (std + 1e-6)
    X_test_n = (X_test - mean) / (std + 1e-6)
    y_train_n = (y_train - mean) / (std + 1e-6)
    y_val_n = (y_val - mean) / (std + 1e-6)
    y_test_n = (y_test - mean) / (std + 1e-6)

    # build models
    rnn = build_simple_rnn(X_train_n.shape[1:])
    lstm = build_lstm(X_train_n.shape[1:])
    # callbacks
    cb = [keras.callbacks.EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)]
    # train (kept small to run quickly)
    print("    Training SimpleRNN...")
    rnn_hist = rnn.fit(X_train_n, y_train_n, validation_data=(X_val_n, y_val_n),
                      epochs=MAX_EPOCHS, batch_size=BATCH_SIZE, callbacks=cb, verbose=1)
    print("    Training LSTM...")
    lstm_hist = lstm.fit(X_train_n, y_train_n, validation_data=(X_val_n, y_val_n),
                       epochs=MAX_EPOCHS, batch_size=BATCH_SIZE, callbacks=cb, verbose=1)
```

```

# evaluate
rnn_eval = rnn.evaluate(X_test_n, y_test_n, verbose=0)
lstm_eval = lstm.evaluate(X_test_n, y_test_n, verbose=0)
print(f" RNN test (mse, mae): {rnn_eval}")
print(f" LSTM test (mse, mae): {lstm_eval}")
# predictions for last 30 test samples
nplot = min(30, len(X_test_n))
preds_rnn_n = rnn.predict(X_test_n[-nplot:]).squeeze()
preds_lstm_n = lstm.predict(X_test_n[-nplot:]).squeeze()
preds_rnn = preds_rnn_n * (std + 1e-6) + mean
preds_lstm = preds_lstm_n * (std + 1e-6) + mean
true = y_test[-nplot:]
# dates for those test samples if available
dates = None
if isinstance(full_series.index, pd.DatetimeIndex):
    total_samples = len(full_series) - LOOKBACK
    test_start = int(len(X_train) + len(X_val))
    idx_from = LOOKBACK + test_start
    dates = full_series.index[idx_from: idx_from + len(X_test)][-nplot:]

# plot
plt.figure(figsize=(9,4))
plt.plot(true, label='true')
plt.plot(preds_rnn, label='rnn_pred')
plt.plot(preds_lstm, label='lstm_pred')
plt.title(f"{city} - last {nplot} test days (true vs preds)")
plt.legend()
plt.tight_layout()
plt.show()

# save models
rnn_path = models_dir / f"{city.replace(' ', '_')}_rnn.keras"
lstm_path = models_dir / f"{city.replace(' ', '_')}_lstm.keras"
rnn.save(rnn_path)
lstm.save(lstm_path)

results[city] = {
    "rnn_eval": rnn_eval,
    "lstm_eval": lstm_eval,
    "rnn_model_path": str(rnn_path),
    "lstm_model_path": str(lstm_path)
}

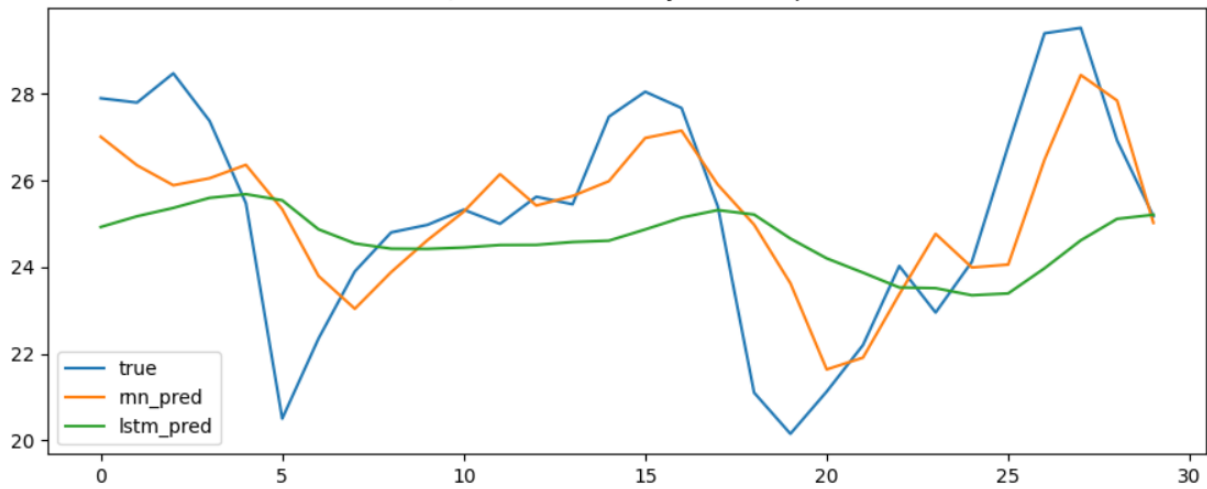
```

```

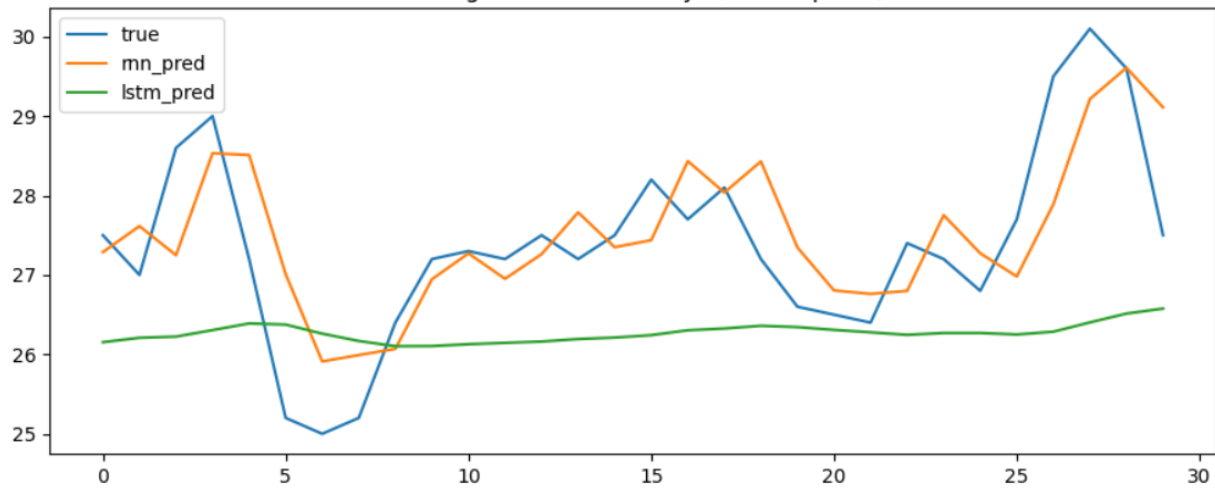
# summary
if results:
    df_summary = pd.DataFrame([
        {
            "city": c,
            "rnn_mse": r["rnn_eval"][0],
            "rnn_mae": r["rnn_eval"][1],
            "lstm_mse": r["lstm_eval"][0],
            "lstm_mae": r["lstm_eval"][1],
            "rnn_model": r["rnn_model_path"],
            "lstm_model": r["lstm_model_path"]
        }
        for c,r in results.items()
    ])
    print("\n=== Training summary ===")
    display(df_summary)
    print(f"Models saved under:", models_dir)
else:
    print("No city processed (dữ liệu không đủ hoặc không tìm thấy series).")

```

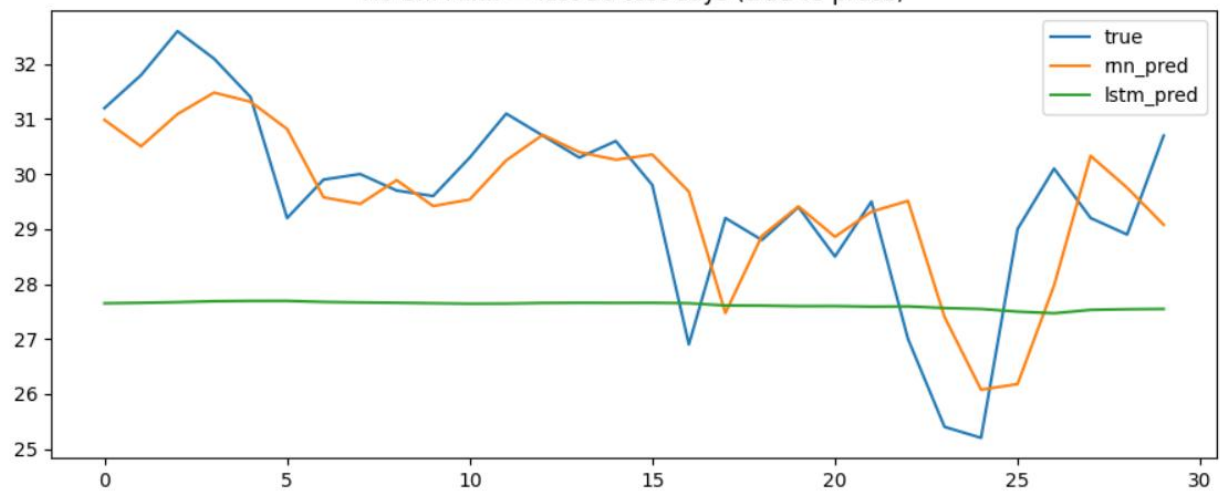
Hà Nội — last 30 test days (true vs preds)



Đà Nẵng — last 30 test days (true vs preds)



Hồ Chí Minh — last 30 test days (true vs preds)



=== Training summary ===

	city	rnn_mse	rnn_mae	lstm_mse	lstm_mae	rnn_model	lstm_model
0	Hà Nội	0.137045	0.281137	0.265303	0.414643	D:\PTIT\20251\IntSys\weather_models\Hà_Nội_rnn...	D:\PTIT\20251\IntSys\weather_models\Hà_Nội_lstm...
1	Đà Nẵng	0.118852	0.252820	0.219370	0.383693	D:\PTIT\20251\IntSys\weather_models\Đà_Nẵng_rnn...	D:\PTIT\20251\IntSys\weather_models\Đà_Nẵng_lstm...
2	Hồ Chí Minh	0.318270	0.400382	2.104168	1.365950	D:\PTIT\20251\IntSys\weather_models\Hồ_Chí_Minh_rnn...	D:\PTIT\20251\IntSys\weather_models\Hồ_Chí_Minh_lstm...

Models saved under: D:\PTIT\20251\IntSys\weather_models

2.4.

[16]: # Nguyễn Hoàng Tùng

```
import os
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import mean_absolute_error, mean_squared_error
import tensorflow as tf
from tensorflow import keras

# =====
# 🌟 Cấu hình
# =====
DATA_DIR = Path(r"D:\PTIT\20251\IntSys") # Thư mục chứa 3 file
MODEL_DIR = Path("weather_models")      # Thư mục chứa các model .keras
LOOKBACK = 14                           # Số ngày để dự đoán ngày kế
TARGET_CITIES = ["Hà Nội", "Đà Nẵng", "Hồ Chí Minh"]

# =====
# 🌟 1. Nạp dữ liệu
# =====
print("Đang tải dữ liệu...")

df_weather = pd.read_csv(DATA_DIR / "df_weather.csv")
weather = pd.read_csv(DATA_DIR / "weather.csv")
precip = pd.read_csv(DATA_DIR / "precipitation_mean_max_min_temp_1901_2020_en.csv")

# Chuyển kiểu ngày
for df in [df_weather, weather]:
    if "date" in df.columns:
        df["date"] = pd.to_datetime(df["date"], errors="coerce")

print("✅ Đã nạp dữ liệu:")
print("- df_weather:", df_weather.shape)
print("- weather:", weather.shape)
print("- precip:", precip.shape)
```

```
def extract_city_series(city):
    city_lower = city.lower()

    # --- UU TIỀN DỮ LIỆU TỪ weather.csv ---
    if "province" in weather.columns:
        mask = weather["province"].astype(str).str.contains(
            "Hà Nội|Hà Nội|Đà Nẵng|Đa Nang|TP.? ?HCM|TP.? ?Hồ Chí Minh|Ho Chi Minh|HoChiMinh|Sai Gon|Saigon",
            case=False, na=False, regex=True
        )
        if mask.any():
            # Lọc đúng theo tên thành phố
            if "hà" in city_lower:
                mask_city = weather["province"].astype(str).str.contains("Hà Nội|Hà Nội", case=False, na=False, regex=True)
            elif "đà" in city_lower or "da" in city_lower:
                mask_city = weather["province"].astype(str).str.contains("Đà Nẵng|Đa Nang", case=False, na=False, regex=True)
            elif "hồ" in city_lower or "chi" in city_lower or "hcm" in city_lower or "sài" in city_lower:
                mask_city = weather["province"].astype(str).str.contains(
                    "Hồ Chí Minh|TP.? ?HCM|TP.? ?Hồ Chí Minh|Ho Chi Minh|HoChiMinh|Sai Gon|Saigon",
                    case=False, na=False, regex=True
                )
            else:
                mask_city = pd.Series(False, index=weather.index)

        df_city = weather.loc[mask_city].copy()
        if len(df_city) > 0:
            df_city = df_city.dropna(subset=["max", "min", "date"])
            df_city["temp_avg"] = (df_city["max"] + df_city["min"]) / 2.0
            df_city = df_city.groupby("date")["temp_avg"].mean().sort_index()
            return df_city
```

```
# --- DỮ LIỆU DỰ PHÒNG TỪ df_weather.csv ---
if "hà" in city_lower:
    mask = df_weather["location.name"].astype(str).str.contains("Hà Nội", case=False, na=False)
    series = df_weather.loc[mask].set_index("date")["day.avgtemp_c"]
elif "đà" in city_lower or "da" in city_lower:
    mask = df_weather["location.name"].astype(str).str.contains("Đà Nẵng", case=False, na=False)
    series = df_weather.loc[mask].set_index("date")["day.avgtemp_c"]
elif "hồ" in city_lower or "chi" in city_lower or "hcm" in city_lower:
    mask = df_weather["location.name"].astype(str).str.contains("Hồ Chí Minh|Ho Chi Minh", case=False, na=False)
    series = df_weather.loc[mask].set_index("date")["day.avgtemp_c"]
else:
    series = pd.Series(dtype=float)

series = series.dropna().sort_index()
if not series.index.is_unique:
    series = series.groupby(series.index).mean()
return series

# Tạo dữ liệu cho các thành phố
city_series = {city: extract_city_series(city) for city in TARGET_CITIES}

for c, s in city_series.items():
    print(f"{c}: {len(s)} bản ghi, {s.index.min()} → {s.index.max()} if len(s)>0 else 'N/A'")
```

```
# =====
# 🌀 3. Tạo dataset
# =====
def make_dataset(series, lookback=LOOKBACK):
    s = series.copy().dropna().sort_index()
    if isinstance(s.index, pd.DatetimeIndex):
        s = s.asfreq('D')
        s = s.interpolate(limit_direction='both')
    arr = s.values.astype(np.float32)
    X, y = [], []
    for i in range(lookback, len(arr)):
        X.append(arr[i-lookback:i])
        y.append(arr[i])
    X = np.array(X)[: , :, None]
    y = np.array(y)
    return X, y, s

# =====
# 🧡 4. Load mô hình
# =====
model_files = list(MODEL_DIR.glob("*.keras"))
if not model_files:
    raise FileNotFoundError(f"Không tìm thấy model trong {MODEL_DIR}")

loaded_models = {}
for file in model_files:
    model = keras.models.load_model(file)
    model_type = "RNN" if "rnn" in file.stem.lower() else "LSTM"
    city_key = None
    for city in TARGET_CITIES:
        if city.split()[0].lower() in file.stem.lower():
            city_key = city
            break
    if city_key:
        loaded_models.setdefault(city_key, {})[model_type] = model

print("\n✅ Đã load mô hình:")
for c, models in loaded_models.items():
    print(f" - {c}: {list(models.keys())}")
```

```
# =====
# 📊 5. So sánh mô hình
# =====
results = []

for city in TARGET_CITIES:
    if city not in city_series or len(city_series[city]) < 60:
        print(f"⚠️ {city}: không đủ dữ liệu để đánh giá.")
        continue

    series = city_series[city]
    X, y, s = make_dataset(series)
    mean, std = X.mean(), X.std()
    Xn = (X - mean) / (std + 1e-6)

    if city not in loaded_models:
        print(f"⚠️ Không tìm thấy model cho {city}")
        continue

    for model_name, model in loaded_models[city].items():
        preds = model.predict(Xn).squeeze() * (std + 1e-6) + mean
        mae = mean_absolute_error(y, preds)
        mse = mean_squared_error(y, preds)
        results.append({"Thành phố": city, "Mô hình": model_name, "MAE": mae, "MSE": mse})

    plt.figure(figsize=(9,4))
    plt.plot(y[-30:], label="Thực tế", color="black")
    plt.plot(preds[-30:], label=f"{model_name} dự báo", linestyle="--")
    plt.title(f"So sánh dự báo và thực tế - {city} ({model_name})")
    plt.xlabel("Ngày (cuối)")
    plt.ylabel("Nhiệt độ (°C)")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()
```

```
# =====
# 📊 6. Tổng hợp kết quả
# =====
if not results:
    raise RuntimeError("Không có kết quả để vẽ. Kiểm tra dữ liệu và model.")

df_results = pd.DataFrame(results)
print("\nBảng kết quả:")
print(df_results)

plt.figure(figsize=(8,5))
sns.barplot(data=df_results, x="Thành phố", y="MAE", hue="Mô hình")
plt.title("So sánh sai số MAE giữa các mô hình")
plt.tight_layout()
plt.show()

plt.figure(figsize=(8,5))
sns.barplot(data=df_results, x="Thành phố", y="MSE", hue="Mô hình")
plt.title("So sánh sai số MSE giữa các mô hình")
plt.tight_layout()
plt.show()

out_path = DATA_DIR / "compare_results.xlsx"
df_results.to_excel(out_path, index=False)
print(f"\n✅ Kết quả đã lưu: {out_path}")
```

Đang tải dữ liệu...

✅ Đã nạp dữ liệu:

- df_weather: (26018, 34)
- weather: (181960, 10)
- precip: (999, 7)

Hà Nội: 4549 bản ghi, 2009-01-01 00:00:00 → 2021-06-18 00:00:00

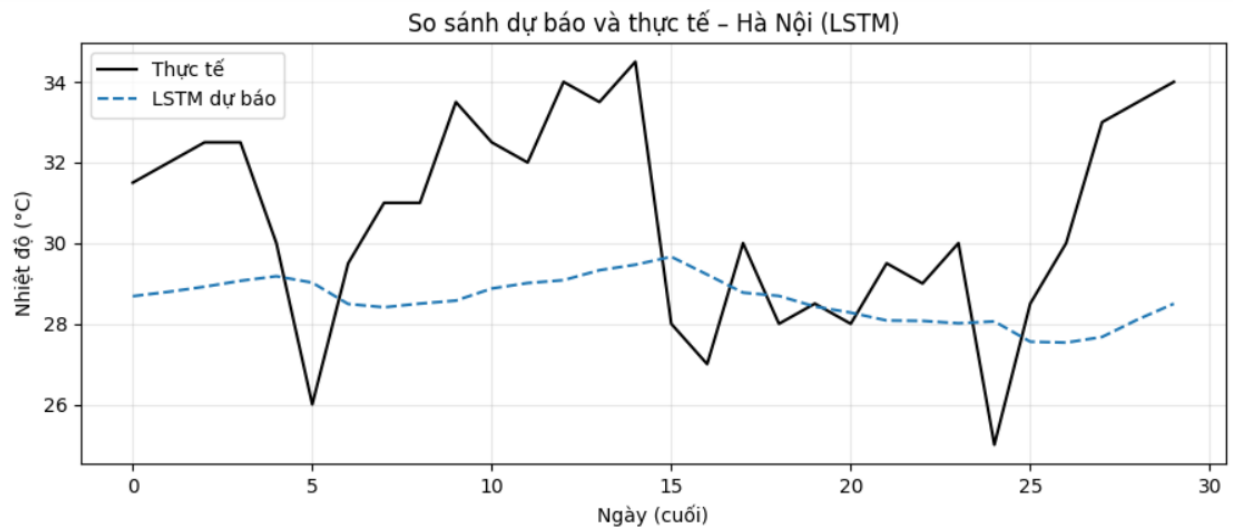
Đà Nẵng: 409 bản ghi, 2024-04-21 00:00:00 → 2025-06-04 00:00:00

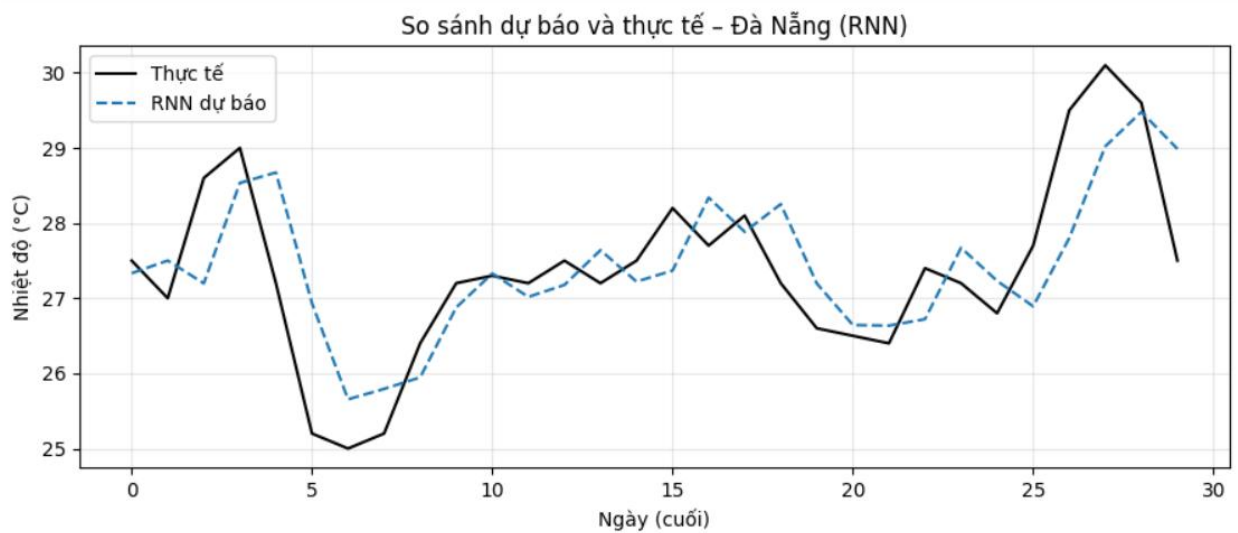
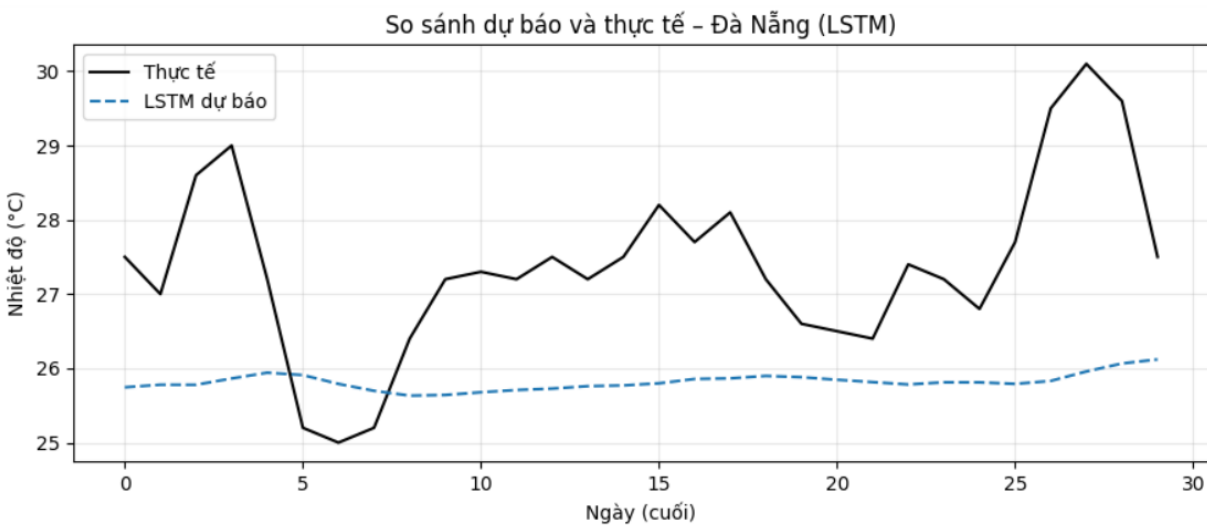
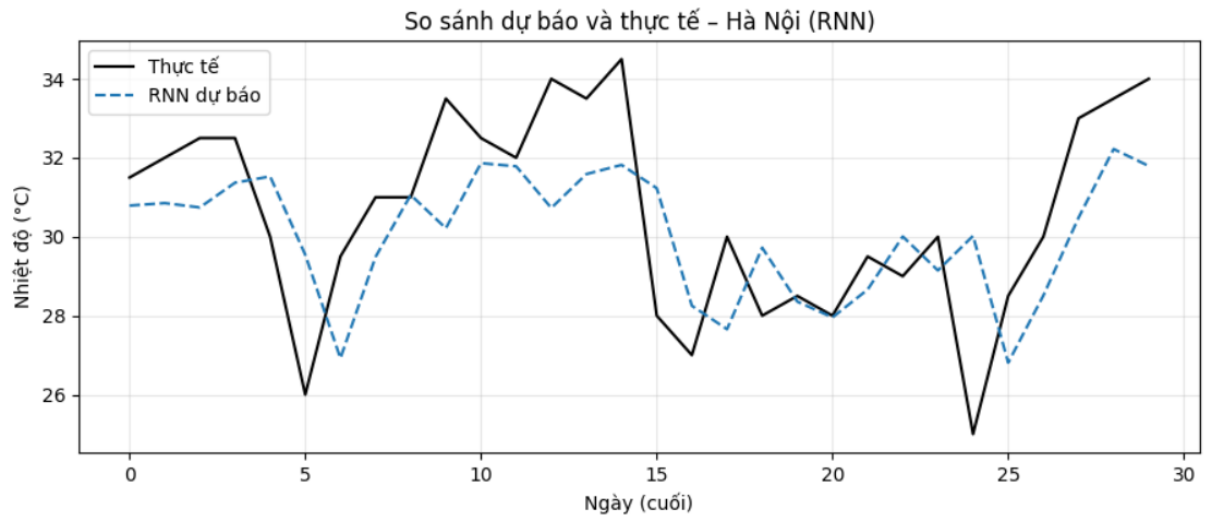
Hồ Chí Minh: 4549 bản ghi, 2009-01-01 00:00:00 → 2021-06-18 00:00:00

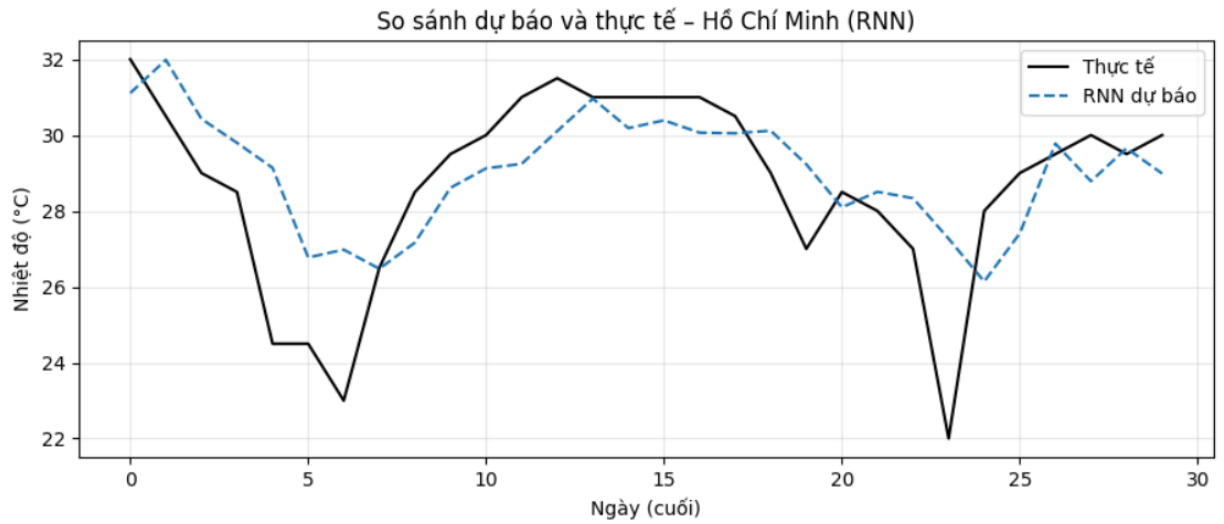
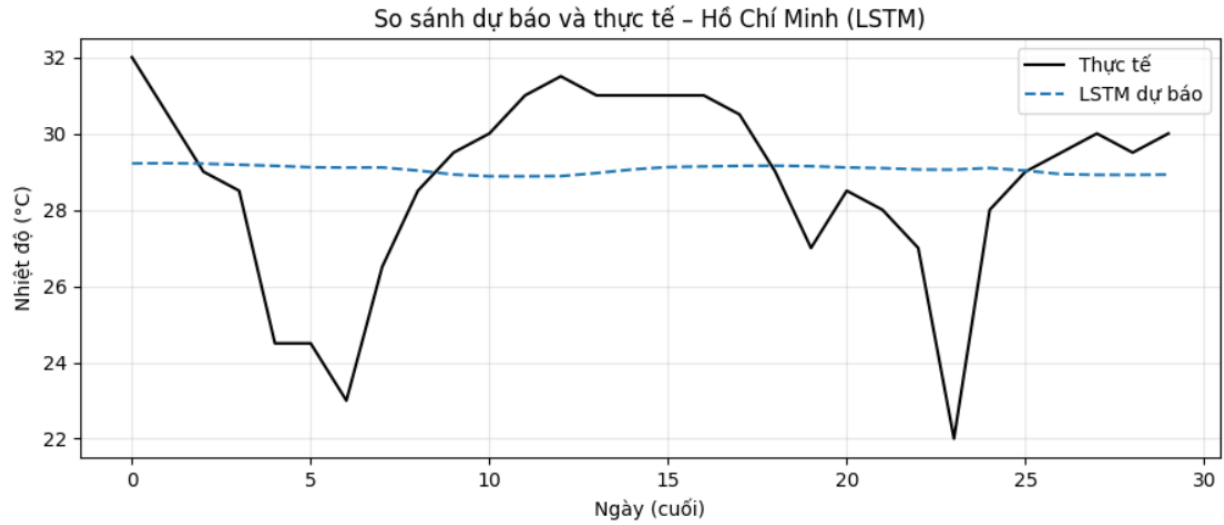
✅ Đã load mô hình:

- Hà Nội: ['LSTM', 'RNN']
- Hồ Chí Minh: ['LSTM', 'RNN']
- Đà Nẵng: ['LSTM', 'RNN']

142/142 ————— 1s 3ms/step

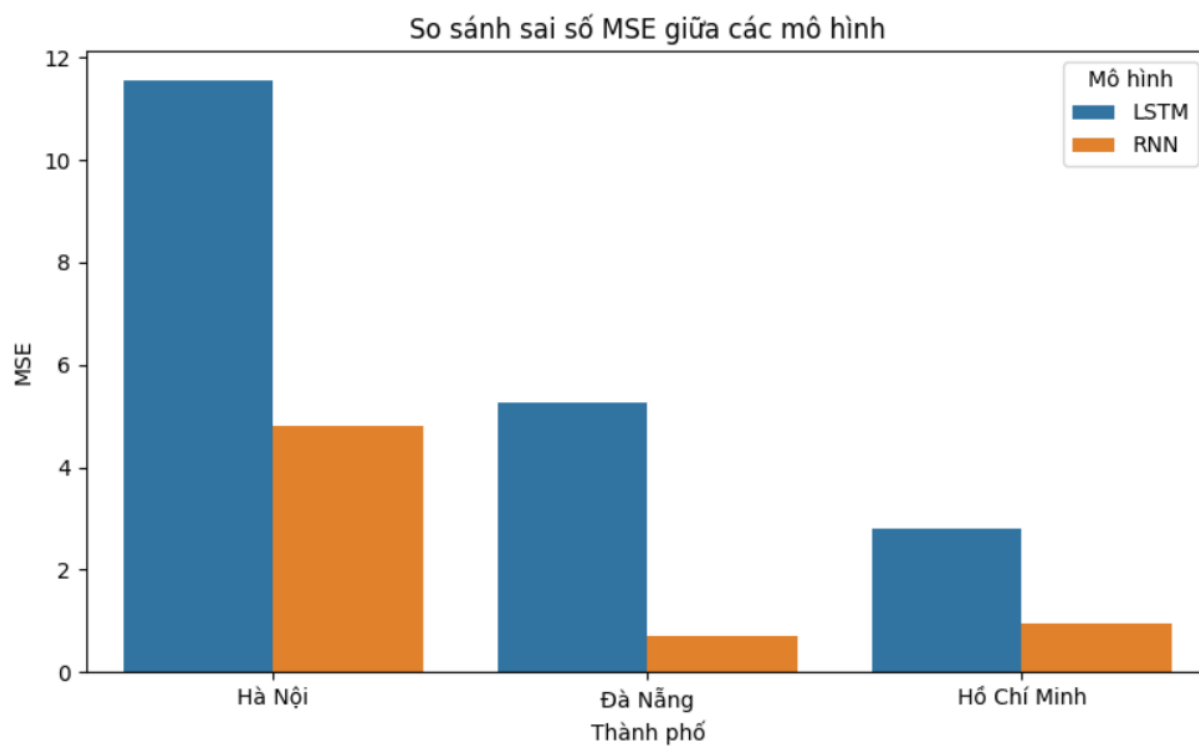
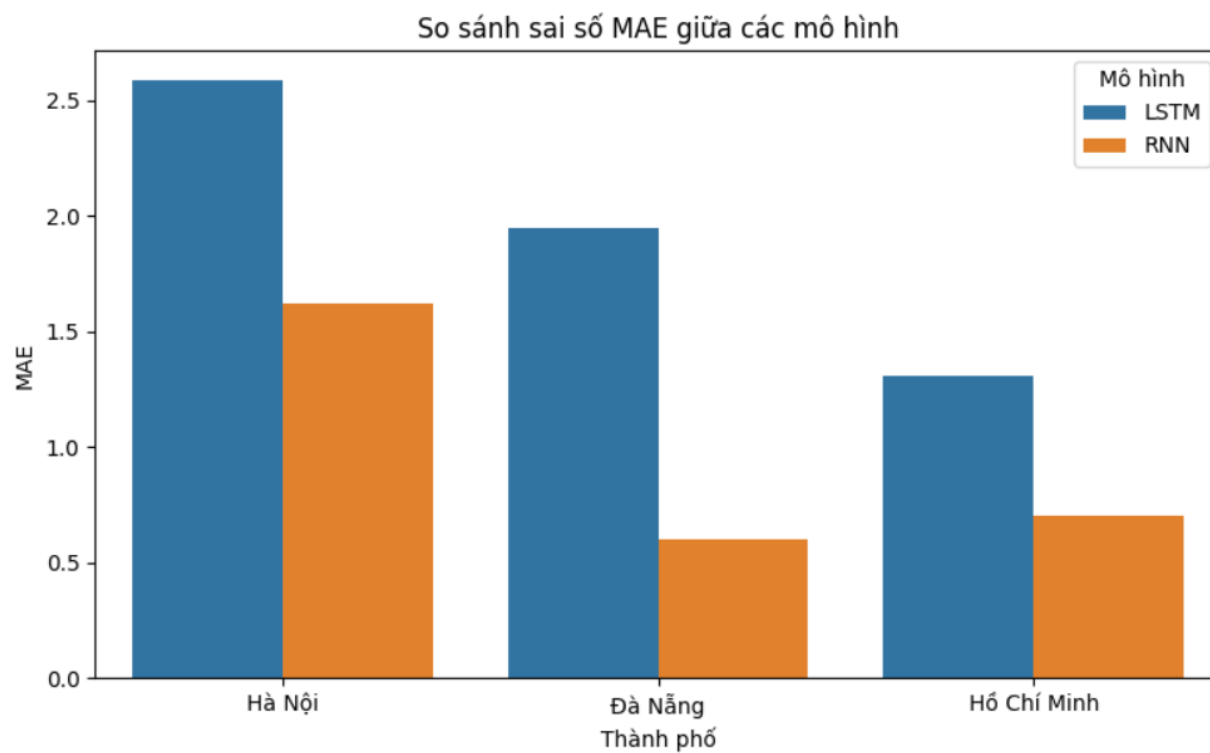






Bảng kết quả:

	Thành phố	Mô hình	MAE	MSE
0	Hà Nội	LSTM	2.587591	11.556654
1	Hà Nội	RNN	1.621012	4.815606
2	Đà Nẵng	LSTM	1.945312	5.263058
3	Đà Nẵng	RNN	0.600730	0.702077
4	Hồ Chí Minh	LSTM	1.304437	2.800327
5	Hồ Chí Minh	RNN	0.705614	0.940882



2.5.

* Mô hình RNN

Loss:

- Cả *train_loss* và *val_loss* đều giảm dần theo số epoch, cho thấy mô hình học được quy luật của dữ liệu và dần cải thiện trong quá trình huấn luyện.
- Đường *val_loss* nằm thấp hơn *train_loss*, có thể do một hoặc nhiều nguyên nhân sau:
 - Regularization (dropout) tương đối mạnh khiến *train_loss* tăng lên.
 - Tập validation nhỏ hơn hoặc dễ hơn so với tập huấn luyện.
 - Dữ liệu huấn luyện có nhiều nhiễu (noise), khiến việc tối ưu trở nên khó hơn.
- Không xuất hiện dấu hiệu overfitting (không có giai đoạn *val_loss* tăng trở lại ở cuối quá trình huấn luyện).
→ Kết luận: Mô hình tổng quát hóa tốt và đạt độ ổn định cao trên tập validation.

MAE:

- Cả *train_mae* và *val_mae* đều giảm dần theo số epoch, thể hiện độ chính xác của mô hình được cải thiện liên tục.
- *val_mae* nhỏ hơn *train_mae*, phù hợp với nhận xét từ biểu đồ *loss*.
- Không xuất hiện dao động lớn hoặc xu hướng tăng ngược ở đường *validation*, cho thấy quá trình huấn luyện ổn định.
→ Kết luận: Mô hình RNN học tốt, ổn định và không bị overfitting.

* Mô hình LSTM

Loss:

- Cả *train_loss* và *val_loss* đều giảm đều theo thời gian, chứng tỏ mô hình học tốt hơn qua từng epoch.
- Đáng chú ý, *val_loss* luôn thấp hơn *train_loss* — hiện tượng hiếm gặp nhưng có thể xảy ra khi:
 - Regularization mạnh (dropout, L2) khiến *train_loss* tăng lên.
 - Tập validation có quy mô nhỏ hơn hoặc chứa dữ liệu dễ dự đoán hơn.
 - Dữ liệu huấn luyện chứa nhiều nhiễu hơn tập validation.

- Không có dấu hiệu overfitting, *val_loss* không tăng trở lại mà còn giảm nhẹ ở giai đoạn cuối.
→ Kết luận: Quá trình tối ưu ổn định, mô hình hội tụ tốt và không bị quá khớp.

MAE:

- Cả *train_mae* và *val_mae* đều giảm liên tục, cho thấy sai số tuyệt đối trung bình giảm theo thời gian.
- *val_mae* thấp hơn *train_mae*, nhất quán với xu hướng từ biểu đồ *loss*.
- Không xuất hiện dao động mạnh, chứng minh mô hình học ổn định.
→ Kết luận: Mô hình LSTM cải thiện khả năng dự báo liên tục và đạt mức sai số thấp, thể hiện năng lực khái quát hóa tốt.

* So sánh

- Cả hai mô hình RNN và LSTM đều huấn luyện ổn định, không có dấu hiệu overfitting.
- LSTM thường đạt *val_loss* và *val_mae* thấp hơn, phản ánh khả năng học phụ thuộc dài hạn tốt hơn RNN.
- RNN vẫn hoạt động ổn định và có thể đủ hiệu quả với dữ liệu chuỗi ngắn hoặc ít nhiễu hơn.